ADAPTIVA

Adaptive learning and accessibility for everyone

Theme: Accessibility & Inclusive Technology

Website	**https://adaptiva-flame.vercel.app/**
GitHub	**https://github.com/skandakn/Adaptiva**

Final Hackathon Documentation

Same knowledge. A learning experience adapted to the learner.

Adaptive learning and accessibility for everyone

Theme: Accessibility & Inclusive Technology

Hosted website: https://adaptiva-flame.vercel.app/

GitHub repository: https://github.com/skandakn/Adaptiva

1. Executive Summary

Adaptiva is an accessibility-first learning platform that reshapes educational content around the learner instead of forcing every learner into the same presentation style. It supports adaptive reading, audio, visual explanation, translation, progress tracking, and learner preferences so that people can approach the same knowledge in a more comfortable format.

The project is designed for learners who may need simplified language, better spacing, read-aloud support, translation, focus guidance, or visual explanation. It also includes teacher-facing and learner-facing views, authentication, persistence, and AI-assisted adaptation routes.

2. Problem Statement

The hackathon problem asks for a solution to digital exclusion caused by visual, hearing, motor, cognitive, language, and literacy barriers. Adaptiva addresses this by adapting the content layer itself. Instead of presenting one rigid interface to every user, it offers multiple accessible paths to the same learning material. This reduces the friction created by dense text, fast lectures, unfamiliar vocabulary, language barriers, and low digital confidence.

3. Solution to the Problem Statement

Adaptiva turns learning material into an accessibility-first experience instead of a fixed page. A learner can open a lesson, paste text, upload an image, join a live lecture, or upload recorded video, and Adaptiva adapts the content into simpler language, step-by-step explanations, spoken output, visual concept maps, translated text, and saved notes. Its AI layer helps understand the source material, identify concepts, and reframe the same knowledge in different formats, while the accessibility layer applies preferences such as OpenDyslexic reading, font size, spacing, focus guidance, and language direction. For scanned documents, the app can use image understanding with OCR fallback; for live lectures and recorded video, it produces transcripts and accessible notes; and for study questions, Ask Adaptiva answers in a learner-centered way. This directly addresses Accessibility & Inclusive Technology by reducing cognitive, language, and format barriers without asking the learner to change the content themselves. Adaptiva makes one idea available in multiple ways so more people can learn with dignity, clarity, and control.

4. Application Description

Adaptiva is built around one principle: same knowledge, a learning experience adapted to the learner. The interface combines a public landing page, onboarding, adaptive learning workspace, live lecture mode, recorded video mode, progress tracking, a teacher dashboard, an accessibility settings panel, and an AI tutor experience. The platform can simplify content, read it aloud, translate it, reorganize it into steps or concept maps, and store the learner's preferences and learning history when persistence is configured.

5. Prompts Implemented

The repository contains several real prompt templates inside the AI service layer. They are used to shape adaptation, translation, figure generation, and chatbot behavior.

Prompt name	Where it is used	Template or instruction shape	Output
Accessible rewrite	`simplifyText()`	"Rewrite educational content in accessible plain language."	Simplified learner-friendly text
Low-load summary	`summarizeContent()`	"Summarize educational content for a learner who benefits from low cognitive load."	Short summary
Sequential steps	`explainStepByStep()`	"Break the concept into numbered sequential learning steps."	Step-by-step explanation
Ask Adaptiva tutor	`askAdaptivaChat()`	"You are Ask Adaptiva, a general educational assistant... answer direct educational questions first."	Chat reply for learners
Transcript notes	`generateNotesFromTranscript()`	"Generate clear, structured educational notes from a video transcript."	Notes from transcript text
Image understanding	`analyzeUploadedImage()`	"You are an accessibility-first learning assistant. Analyze only the uploaded image."	Image explanation or OCR-based fallback
Figure JSON generator	`generateWithAI()` in `lib/figure-service.ts`	"Convert educational text into a structured JSON figure specification. Return ONLY valid JSON."	FigureSpec JSON
UI translation batcher	`translateUiStrings()`	"Translate Adaptiva UI text... preserve names, acronyms, numbers, URLs, and code-like tokens."	Translated interface strings
Mind map translation	`translateMindMap()`	"Translate each mind-map label into the target language and keep the same ids."	Localized concept map
Quiz generation	`generateQuiz()`	"Create three accessible quiz questions and answers from the learning content."	Short quiz

6. Core Functionality

Feature	What it does	Accessibility value	Status
Adaptive learning workspace	Adapts a lesson into Original, Simplified, Focus, Audio, and Visual modes.	Gives learners multiple ways to reach the same concept.	Implemented
Reading Mode	Controls font, text size, spacing, focus guide, audio speed, and read-aloud behavior.	Supports readable layout, reduced strain, and sentence highlighting.	Implemented
OpenDyslexic and spacing preferences	Applies user preferences from `/api/profile` and local storage.	Makes reading more comfortable and personalized.	Implemented
Translation support	Translates visible UI text and lesson content for English, Hindi, Kannada, Urdu, and Tamil.	Reduces language barriers and supports RTL Urdu layout.	Implemented with OpenAI/deterministic fallback
OCR and image adaptation	Accepts uploaded images or scanned documents and explains them.	Supports text extraction from scans and image-based learning.	Implemented with OpenAI vision or local OCR fallback
Live lecture mode	Uses browser speech recognition or demo speech fallback to capture a live transcript and save lecture notes.	Helps learners review spoken content and keep accessible notes.	Implemented
Recorded video mode	Uploads video and sends it to Groq Whisper for timestamped transcription.	Converts video speech into readable segments.	Implemented; summary text in the UI is fallback/demo content

Feature	What it does	Accessibility value	Status
Ask Adaptiva	Provides an AI learning assistant through `/api/chat`.	Lets learners ask for examples, summaries, steps, and simpler explanations.	Implemented with Groq fallback behavior
Tutor page	Shows a demo conversation for the sample lesson.	Demonstrates adaptive Q&A without claiming live AI on that page.	Demo-only UI
Text-to-Figure	Converts educational text into a structured visual explanation with SVG/table rendering.	Adds another way to understand complex concepts.	Implemented with AI and deterministic fallback
Progress tracking	Stores sessions, progress, notes, and charts.	Lets learners see activity without turning learning into noisy gamification.	Implemented with Supabase or demo store
Authentication	Uses Clerk sign-in, sign-up, and route protection when configured.	Protects private learner data and saved workspaces.	Implemented
Persistence	Uses Supabase/PostgreSQL tables with row-level security when configured.	Keeps each user's data isolated.	Implemented when Supabase is configured; demo store otherwise
Teacher dashboard	Shows educator-facing content tooling and concept insights.	Supports accessible material creation and analysis.	Implemented

Note: the landing page and schema mention PDF/document input, but the current codebase does not include a dedicated PDF parsing route. The implemented content flows are text, image, live audio, and recorded video, plus demo copies of those experiences.

7. User Journey / How it Works

Learner -> selects or uploads learning material -> Adaptiva understands the content -> AI and accessibility rules shape the output -> content is simplified, translated, narrated, mapped, or transcribed -> learner receives an accessible learning experience -> preferences, notes, and progress are stored when persistence is available

8. Accessibility-First Design

- OpenDyslexic and other readable font options are available in Reading Mode.
- Text size, spacing, focus guide, and audio speed are user-controlled.
- The interface uses clear typography, strong contrast, large touch targets, and restrained page density.
- Step-by-step explanations reduce cognitive load.
- Read-aloud support helps learners who prefer listening or need additional reinforcement.
- Translation into Hindi, Kannada, Urdu, and Tamil supports language inclusion.
- Visual concept maps and figures provide a non-text path to understanding.
- Learner preferences are presented as settings, not diagnoses.

9. AI & Intelligence

Adaptiva uses AI to reshape the same educational source in different formats:

- simpler explanations
- summaries
- step-by-step guidance

- quizzes
- concept extraction
- mind maps
- translated content
- contextual figure specifications
- image explanations and OCR-informed responses
- transcript-based notes

The implementation is mixed by design:

- Groq powers the main educational chat and transcript-note generation when `GROQ_API_KEY` is set.
- OpenAI powers image understanding, UI translation, and figure generation when `OPENAI_API_KEY` and the configured provider allow it.
- Deterministic demo fallbacks keep the experience usable when keys are absent.

10. System Architecture

```
Learner
-> Next.js frontend
-> Route handlers in /app/api
-> AI services in /lib
-> Accessibility and adaptation layer
-> Auth layer (Clerk or Supabase auth, depending on configuration)
-> Persistence layer (Supabase/PostgreSQL or demo store)
-> Output: adapted text, audio, visuals, transcripts, notes, progress
```

Layer	What it does
Frontend	Next.js App Router pages and React components for learn, live, video, dashboard, progress, settings, tutor, teacher, architecture, and onboarding.
API/backend	Route handlers for adapt, chat, figure, image-adapt, translate, live-notes, video/process, materials, notes, progress, profile, and dashboard.
AI layer	Groq Responses API for core language tasks, OpenAI Responses API for image and figure tasks, fallback logic for demo mode.
Processing layer	Browser speech recognition, speech synthesis, local OCR fallback, video upload handling, text parsing, and figure rendering.
Accessibility layer	Reading Mode, translation, focus guide, simplified reading, concept maps, and visual explanations.
Auth layer	Clerk sign-in/sign-up plus route protection; Supabase auth when configured.
Database layer	Supabase/PostgreSQL tables with row-level security policies.
External services	Groq, OpenAI, Clerk, Supabase, browser speech APIs, and Tesseract.js CDN fallback for OCR.

11. Key Data Flows

- Text -> /api/adapt -> AI prompt -> simplified, summarized, stepped, translated, or mapped content.
- Image -> /api/image-adapt -> OpenAI vision or local OCR fallback -> accessible explanation.
- Live speech -> browser SpeechRecognition -> transcript -> /api/live-notes -> saved notes and optional material.

- Video -> /api/video/process -> Groq transcription -> timestamped transcript.
- Learning text -> /api/figure -> figure spec -> deterministic SVG/table figure renderer.
- UI text -> /api/translate -> DOM scan and cached translations -> translated interface.

12. AI Chatbot / Ask Adaptiva

Ask Adaptiva is the learner-facing assistant attached to the app shell. It supports direct educational questions and quick actions like explain simply, step-by-step, summarize, and give an example. The live widget talks to `/api/chat`, which in turn uses the chat prompt in `lib/ai-service.ts`. The separate `/tutor` page is intentionally demo-oriented and shows a scripted sample conversation tied to the featured lesson. That distinction keeps the documentation honest: the widget is the real AI chat path, while the tutor page is a showcase/demo experience.

13. Text-to-Figure

The Text-to-Figure page converts educational text into structured visuals. The service picks a figure type, extracts concepts, builds a JSON figure specification, and renders it as a process diagram, cycle, concept map, comparison table, flowchart, timeline, system diagram, or infographic. When the AI provider is unavailable, the app falls back to demo figures or a deterministic sentence-based builder. This makes the feature useful in both live and showcase modes, while keeping the visual explanation grounded in the source text.

14. Security, Privacy & Responsible AI

- Secrets stay server-side in environment variables such as `GROQ_API_KEY`, `OPENAI_API_KEY`, `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`, and `CLERK_SECRET_KEY`.
- Clerk protects sign-in, sign-up, and protected routes when configured.
- Supabase migration files enable row-level security so each authenticated user accesses only their own rows.
- Saved notes, materials, progress, and profile data are scoped to the authenticated user.
- The app uses demo fallbacks when persistence or AI configuration is absent instead of pretending production services are active.
- Prompts restrict outputs to the supplied source material, JSON, or transcript where required.
- Uploaded images and videos are treated as user content for processing; the code does not claim any custom encryption layer beyond the platform and service defaults.

15. Technology Stack

Area	Technologies actually used
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS
UI	Lucide React icons, custom panels/buttons, accessible React components
State and helpers	React hooks, Zod validation, utility helpers
AI / transcription	Groq Responses API, Groq Whisper transcription, OpenAI Responses API
OCR / speech	Browser SpeechRecognition, SpeechSynthesis, Tesseract.js CDN fallback

Area	Technologies actually used
Authentication	Clerk
Database	Supabase / PostgreSQL with RLS
Charts	Recharts
Hosting	Vercel

16. Innovation & Impact

Adaptiva is different because accessibility is not treated as a single toggle. It is the core product behavior. The same lesson can become simpler, spoken, translated, chunked, visualized, or saved, depending on the learner's needs. That approach reduces stigma, respects learner choice, and works across different ability profiles and language backgrounds. The project also bridges text, speech, images, and video in one workspace, which makes it feel practical rather than theoretical.

17. Future Scope

- Broader language coverage.
- Stronger PDF ingestion and document parsing.
- Deeper personalization and analytics.
- More multimodal AI for whiteboard, slide, and diagram inputs.
- Better classroom and cohort workflows for teachers.
- Longer-term learning history beyond demo fallback mode.

18. Demo / Hosted Application

- Website: https://adaptiva-flame.vercel.app/
- GitHub: https://github.com/skandakn/Adaptiva

Judges can explore the landing page, create an accessibility profile, open the learning workspace, try live or recorded media flows, test translation and reading mode, inspect the dashboard and progress views, and open the architecture page for implementation context.

19. Conclusion

Adaptiva aims to make digital learning adapt to the learner instead of forcing every learner to adapt to the same interface. It combines accessibility, AI, translation, transcription, and personalization into one coherent experience built for inclusion.